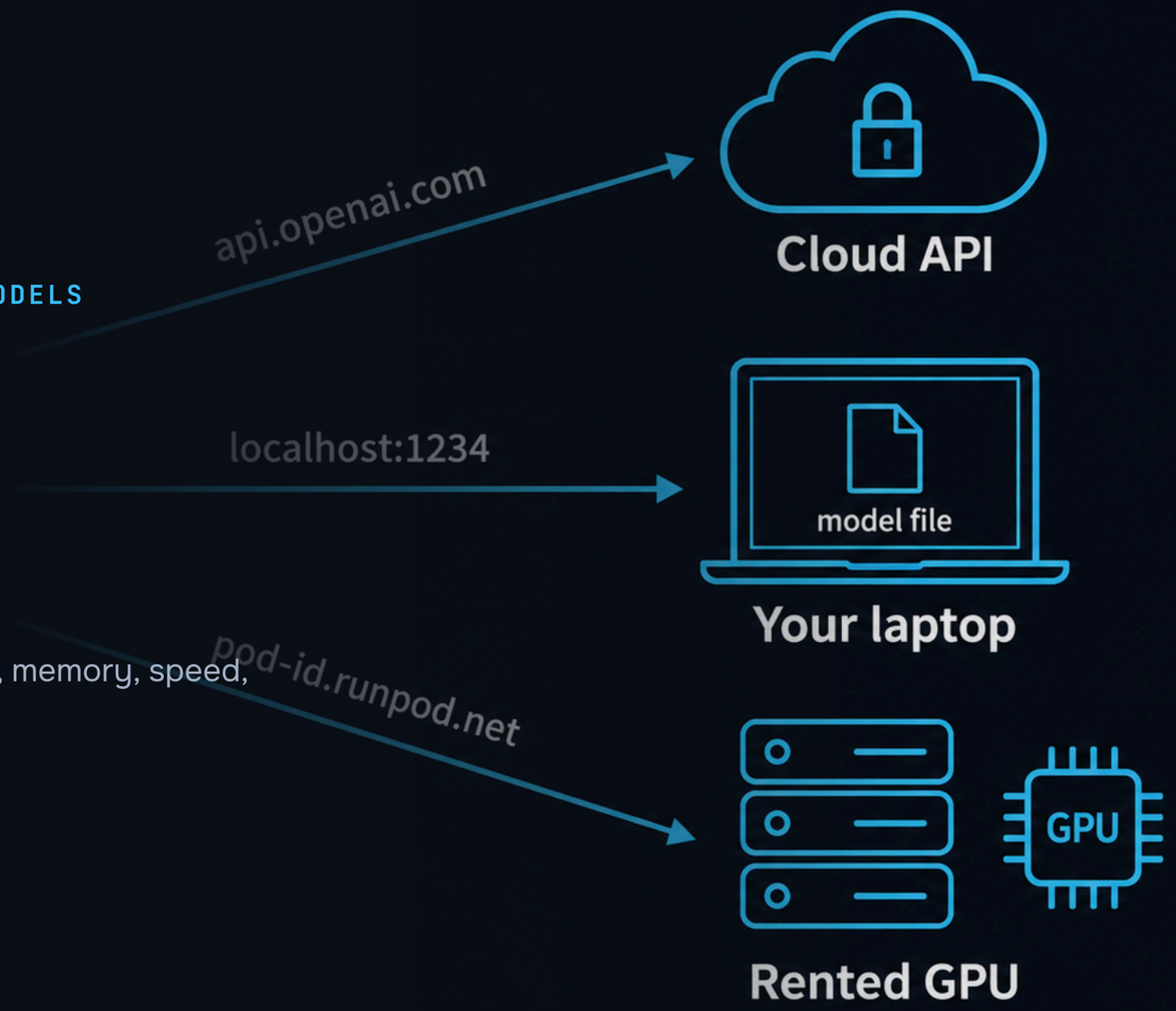
GENAI BASICS • LESSON 10 • MODULE 3: OWN MODELS

Open models

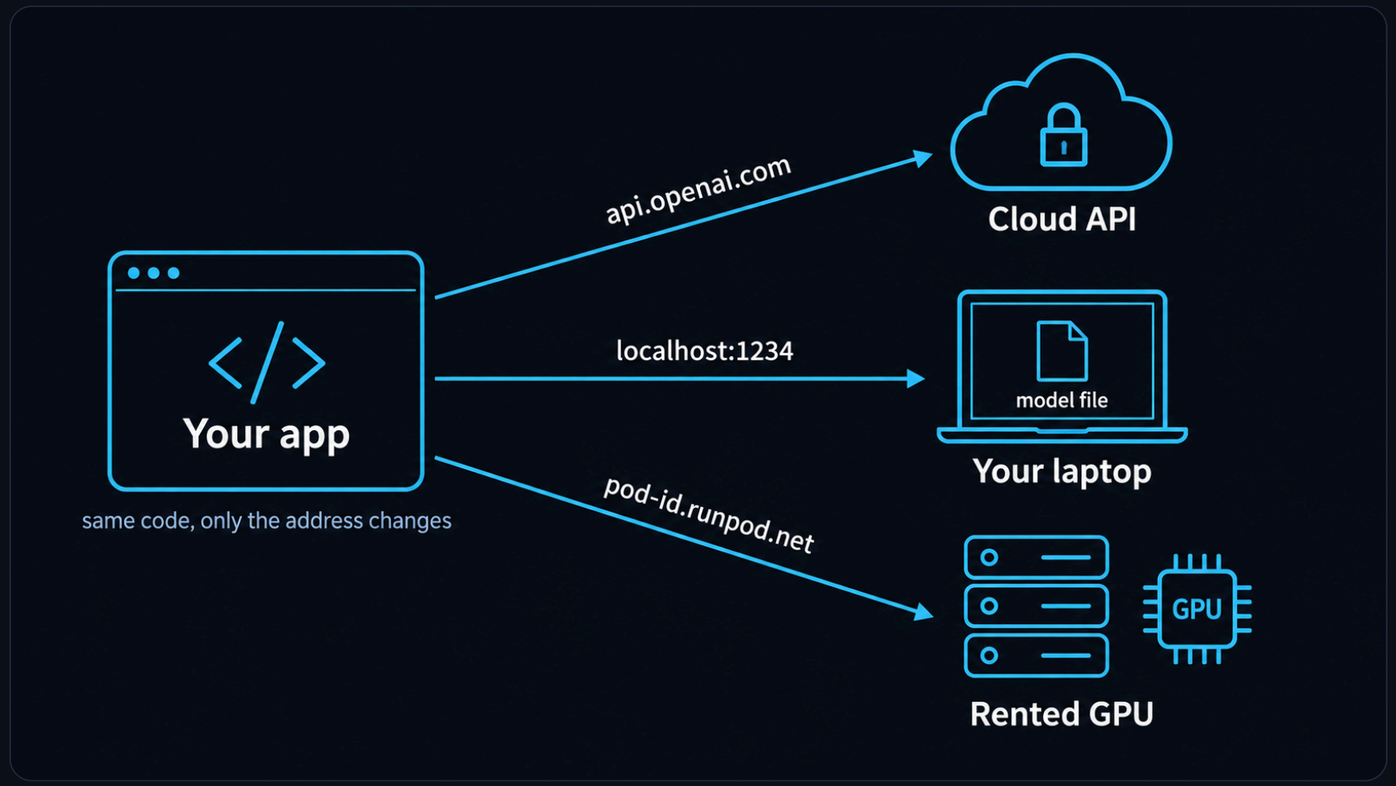
How to run a language model on your laptop and on a rented GPU

Hugging Face, LM Studio, Ollama, RunPod. Model size, memory, speed, price. Data as of October 2026.



THE IDEA

One app, three places where the model runs



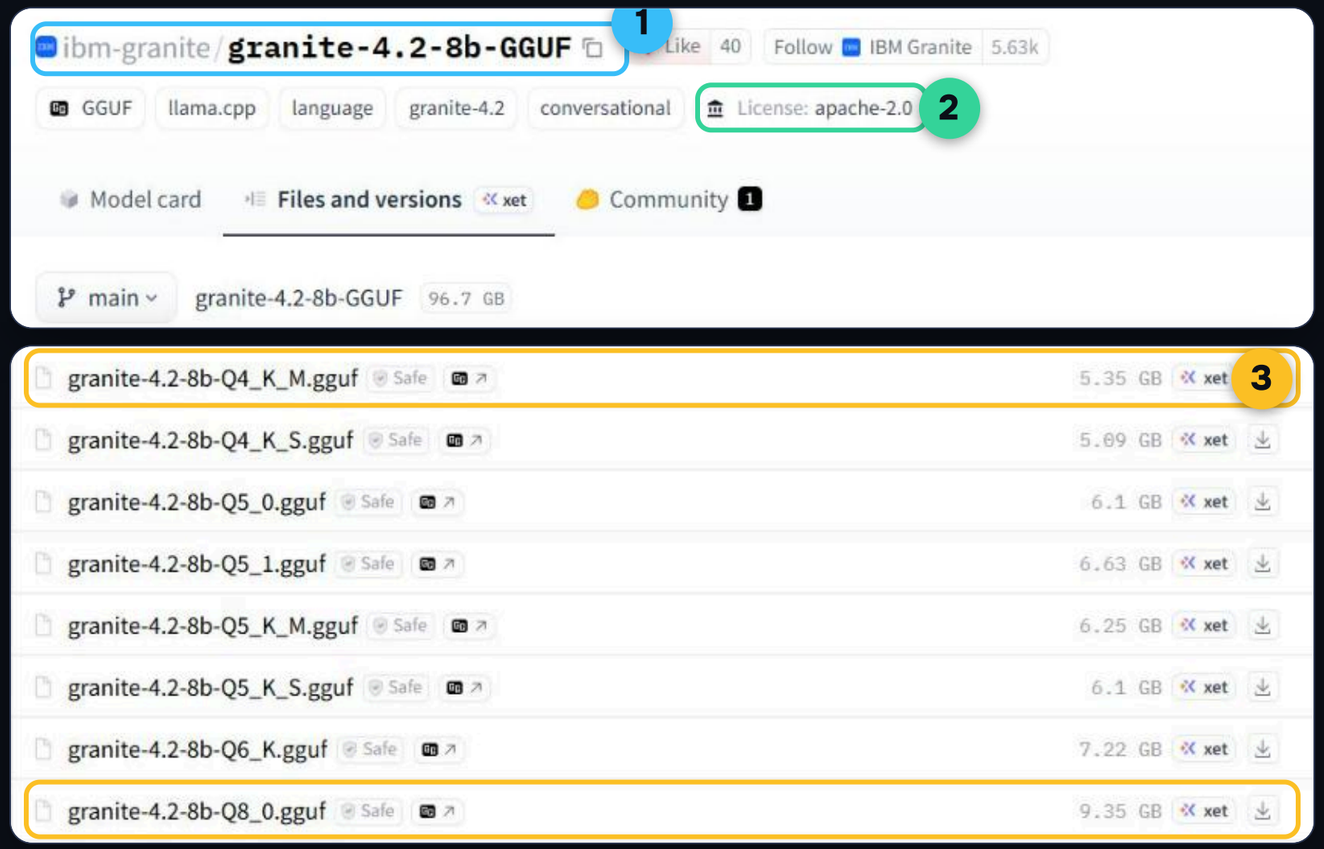
1. **Cloud API**: the vendor runs the model; you pay per token and your data goes to them
2. **Your laptop** (localhost = “this computer”): the model file is yours, no internet needed, you pay with hardware
3. **Rented GPU**: your model on someone else’s powerful machine, paid by the hour

> **Key point:** the app’s code barely changes; only the address it sends the request to does.

OPEN MODEL

An open model is a file you can download, plus its licence

HUGGING FACE · IBM-GRANITE/GRANITE-4.2-8B-GGUF · CAPTURED 16.10.2026



1. **Who made it**: IBM, Granite 4.2 8B. **8B** = 8 billion parameters, the numbers inside the model
2. **Licence** Apache-2.0: fine for work and commercial use
3. **.gguf files** (the format for LM Studio and Ollama): one model in several sizes, here 5.35 to 9.35 GB

Hugging Face is the main “warehouse” of open models. Each has a card: what it does, its limits, its licence.

“Open” ≠ “anything goes”. Llama 4 has its own restrictive licence; MusicGen (L06) is non-commercial only.

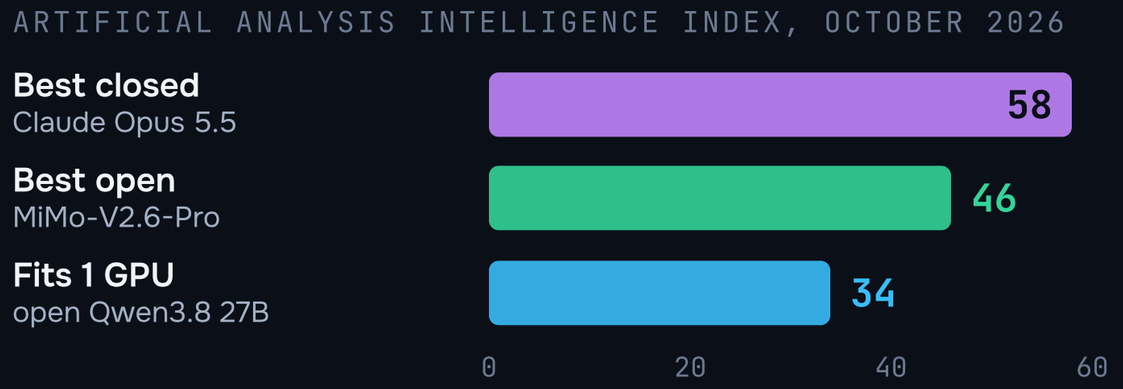
WHY

Your own model: data stays home, but quality trails the leaders

Pros: data never leaves the machine · works offline · no per-token bill · nobody changes the model behind your back

Cons: weaker than the best cloud models · needs memory and a GPU · setup and updates are your job

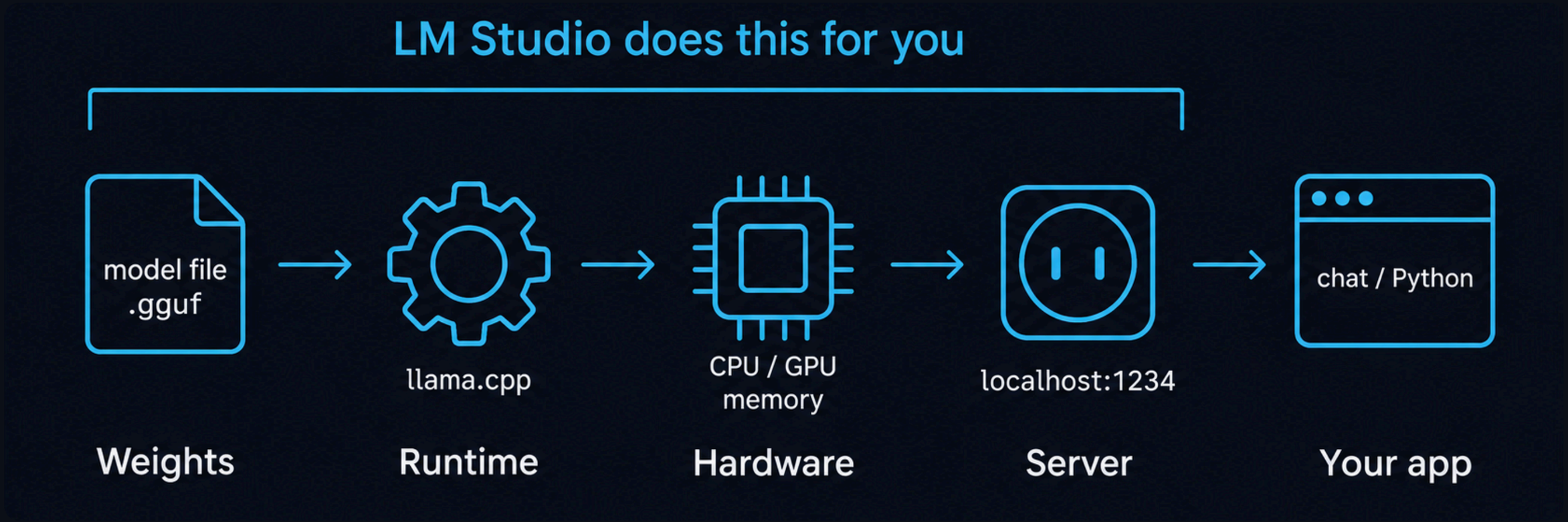
Good for: sorting emails and tickets, drafts, search over your own documents. **Poor at:** hard reasoning, large codebases.



Higher is better. The gap between the best closed and best open model is ≈ 12 points.

HOW IT FITS TOGETHER

Five parts: file, engine, hardware, server, app

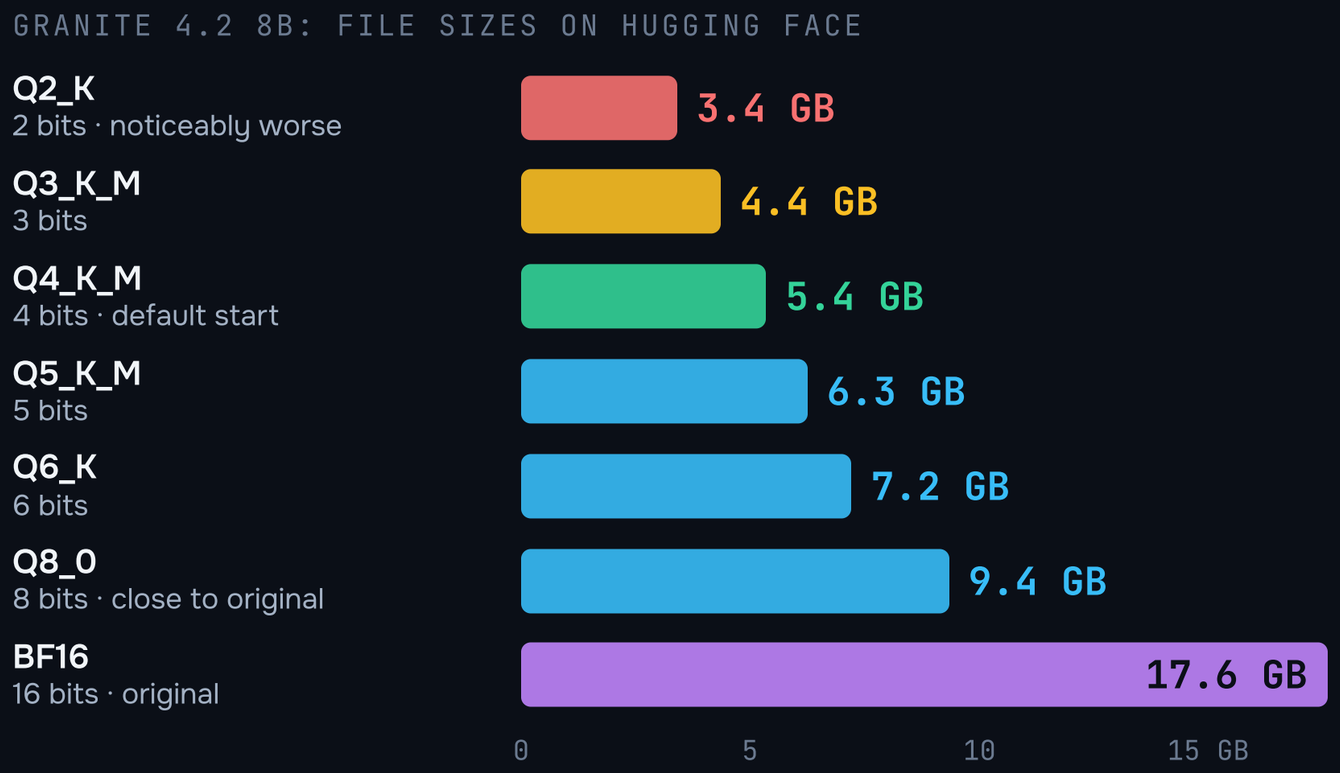


The engine (llama.cpp) reads the model file and computes the answer. **The server** gives it an address, just like a cloud API.

If it is slow, find which part is to blame: file too big? no GPU? Not “the model is bad”.

FILE SIZE

Quantisation: the same model 2–5× smaller, slightly lower quality



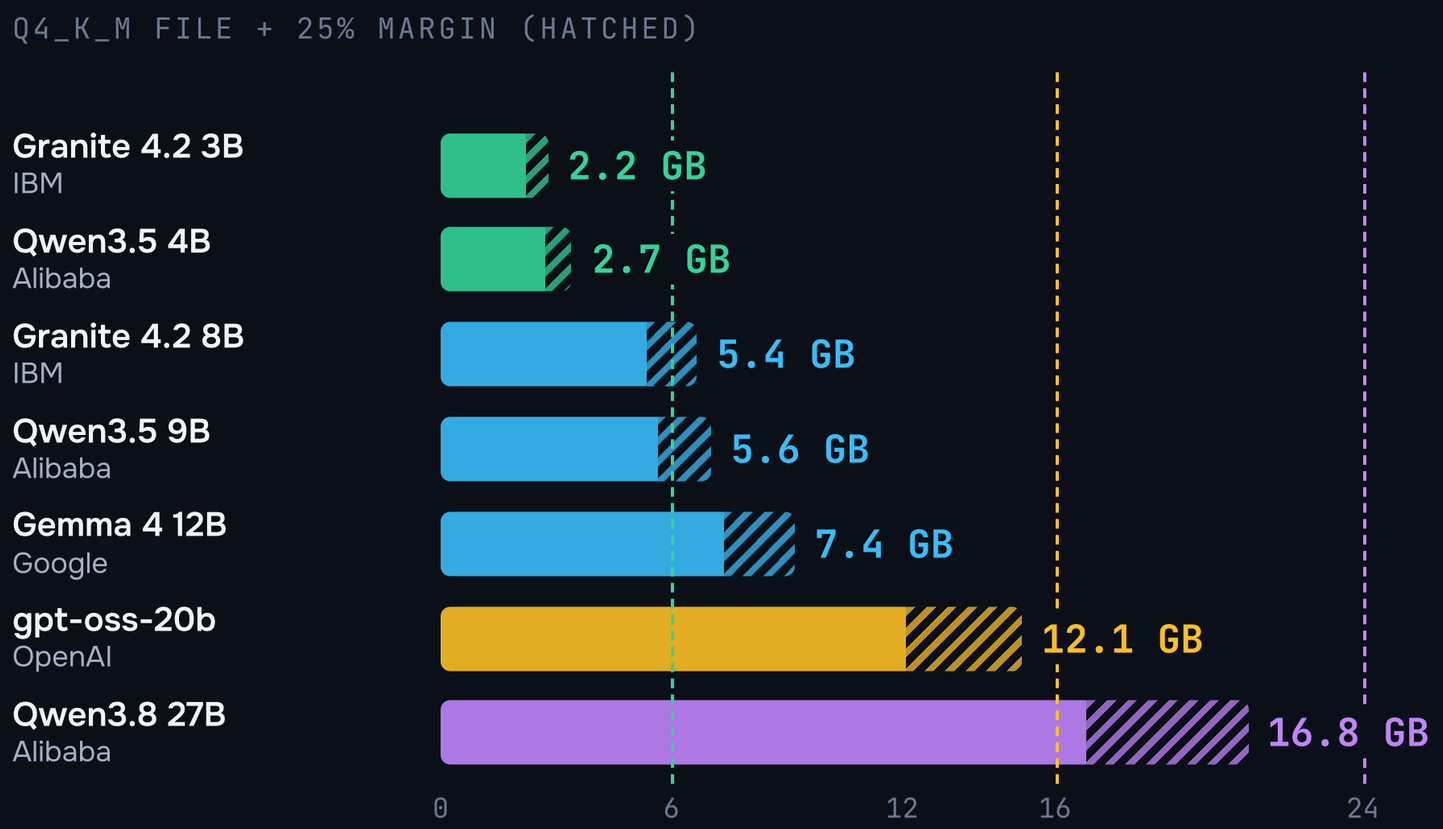
Q4

start with 4 bits; the LM Studio docs say the same

> **How it works:** each number in the model is stored more coarsely, with fewer bits. Like JPEG for photos: a smaller file, slightly fewer details.

WILL IT FIT

The model file plus room for the conversation must fit the GPU memory



+25%

on top of the file: memory for the context, the conversation history (usually 20–30%)

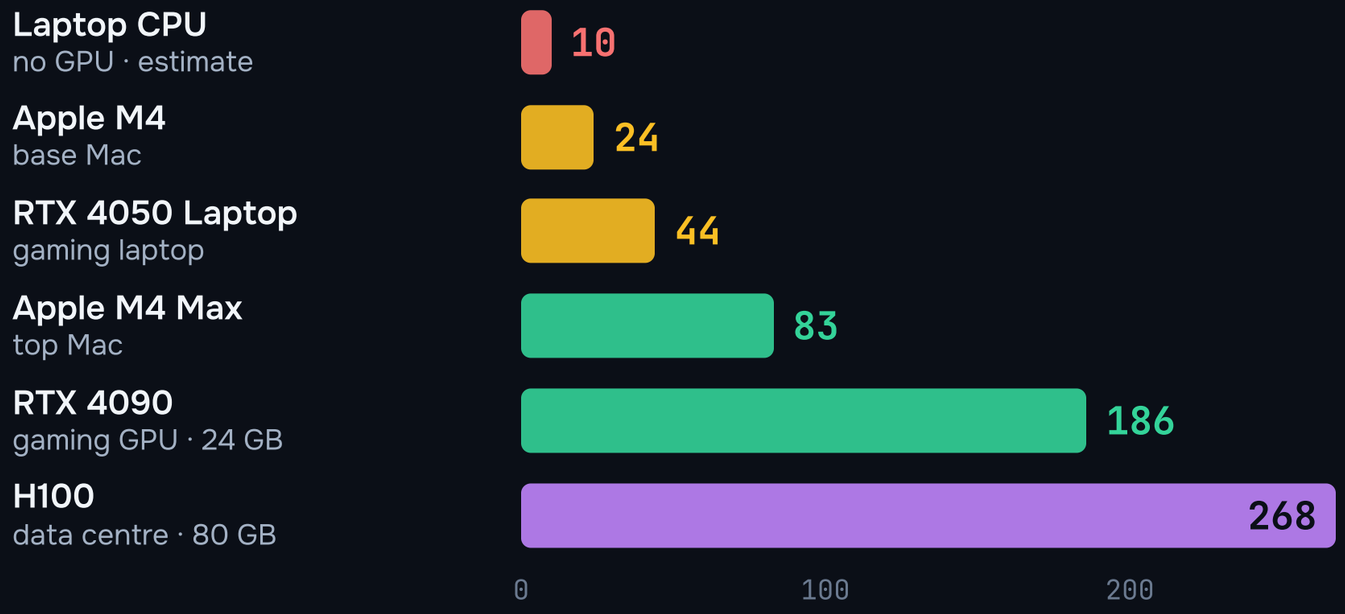
GPU memory (VRAM): **6 GB** → up to 4B · **16 GB** → up to 12B · **24 GB** → up to 27B.

Without a GPU the model lives in system RAM: it fits, but runs slowly. Take 3–4B.

SPEED

A GPU answers 2–25× faster than a CPU

Tokens per second (a token ≈ part of a word), 7B model at 4 bits, llama.cpp. People read ≈ 4–7 tokens per second.



≥ 20 t/s

comfortable: long answers without waiting; 10 t/s is bearable only for short ones

> Numbers are approximate: they depend on the model, conversation length and settings. LM Studio shows your own speed under each answer.

LM Studio: a chat like ChatGPT, but the model runs on your machine

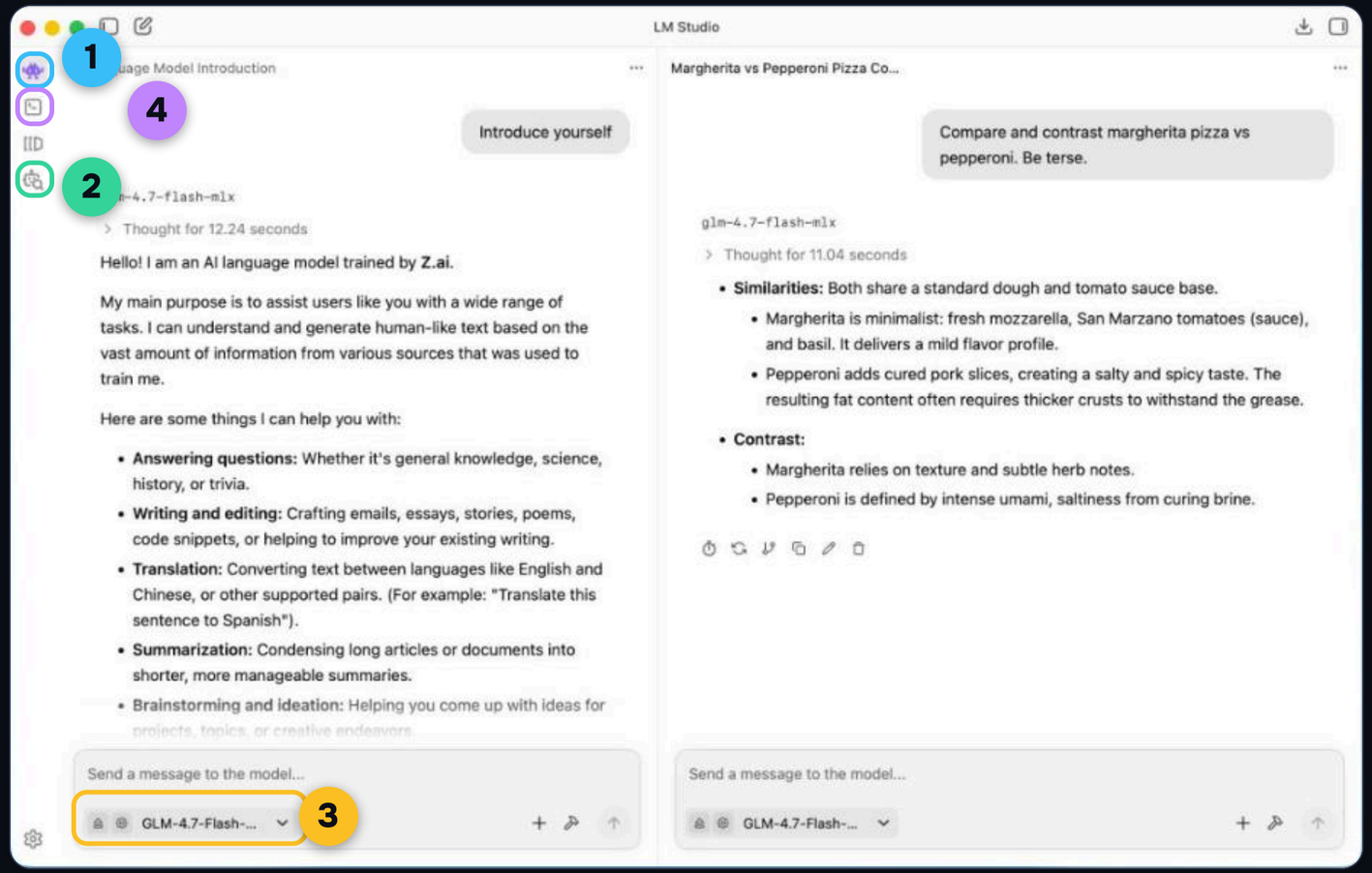


Image: LM Studio / Element Labs, 0.4.0 release blog

1. **Chat** — talk to the model
2. **Discover** — find and download models from Hugging Face
3. **Model picker**: which one to load into memory
4. **Developer** — the local server (slide 11)

> Free, including for work. Windows, Mac (Apple Silicon), Linux. Version 0.4.26.

DEMO 1

Demo: download a small model and ask it a work question

1 Find
Discover → “granite-4.2-3b”

→

2 Download
Q4_K_M · 2.2 GB

→

3 Load
pick the model in Chat

→

4 Ask
the prompt below

→

5 Look
tokens/s under the answer

PROMPT

```
Ticket: “The customer cannot reset their password after
changing phones.”
Return JSON: issue, urgency (low/medium/high), next_step.
```

What to notice: the answer arrives with no internet; the speed in tokens/s is under the answer; memory use rises in Task Manager.

Fallback: if the Wi-Fi is slow, the model is downloaded in advance; demo with that.

LOCAL SERVER

Turn on the server and the model answers your code like a cloud API

PYTHON, THE SAME CODE AS IN L03

```python
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:1234/v1",
    api_key="lm-studio")                # for the cloud: your key

r = client.chat.completions.create(
    model="granite-4.2-3b",
    messages=[{"role": "user", "content": ticket}])
print(r.choices[0].message.content)
```

ONLY THE ADDRESS AND MODEL NAME CHANGE (AND THE KEY)

Cloud (OpenAI)
`https://api.openai.com/v1`
`gpt-5.4-mini`

Laptop (LM Studio)
`http://localhost:1234/v1`
`granite-4.2-3b`

Laptop (Ollama)
`http://localhost:11434/v1`
`granite4.2:3b`

> In LM Studio: **Developer → Start server**. The address `localhost` means “this computer”.

HANDS-ON WITH AN AGENT

A coding agent installs the model for you and measures the speed

PROMPT FOR CLAUDE CODE, CODEX OR ANTIGRAVITY

```
Install Ollama on this computer (official installer from ollama.com)
and run the open model granite4.2:3b.
1. Check the GPU and its memory, system RAM and free disk. Tell me
which model fits.
2. Download the model and ask it: “Briefly: what is quantisation?”
3. Measure the speed (tokens per second) and how much memory the
model uses.
4. Show the local server address and give me an example request in
Python.
5. Expose nothing to the network; the server stays on localhost
only.
```

Ollama — a similar tool that is easy to drive with commands: `ollama run granite4.2:3b`.

The agent checks the hardware, installs the program, downloads the model and reports back: speed, memory, server address.

The agent installs software on your computer: read what it proposes to run and approve each step.

SECURITY

“Local” does not automatically mean “private”: check the whole data path

- **The model computes on your machine**
  the request text goes nowhere
- **Model downloads and updates**
  go to the internet; download from official sources
- **Plug-ins and web search**
  may send data outside
- **Server exposed to the network (not localhost)**
  others can reach the model; turn on Require Authentication

Class rule: server on `localhost` only, models only from Hugging Face by known publishers, client documents only with permission.

> The model licence is part of security too: do not “accidentally” ship a non-commercial model in a product.

MARKET • OCTOBER 2026

Open models by size: what runs where

GPU up to 8 GB		16–24 GB GPU		Server, many GPUs	
Granite 4.2 3B · 8B	Apache-2.0	**Gemma 4 12B**	Apache-2.0	**DeepSeek V4.1**	MIT
Qwen3.5 4B · 9B	Apache-2.0	**gpt-oss-20b**	Apache-2.0	**gpt-oss-120b**	Apache-2.0
Gemma 4 E4B	Apache-2.0	**Qwen3.8 27B**	Apache-2.0	**Llama 4**	custom licence

> Almost everything is Apache-2.0. To start: **Granite 4.2 3B**; smarter but needs a GPU: **Qwen3.5 9B** or **Gemma 4 12B**.

BREAK

5 minutes: grab a coffee while the model downloads

05:00

Next: renting a GPU on RunPod, price, how to choose, homework.

LAPTOP LIMITS

When a laptop is not enough: four signs

Does not fit
the model is bigger than the laptop's memory

Too slow
10,000 emails overnight is too much for a CPU

Many users
one laptop serves one person

Needs 24/7
laptops get closed and taken home

Solution: keep the same model and the same code, but move the computing to a powerful cloud GPU. Only the address changes.

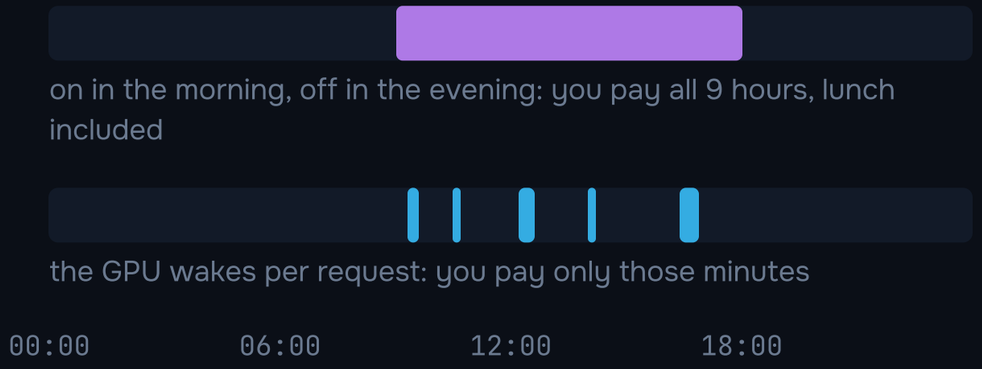
RunPod: a GPU rented with per-second billing, two modes

RUNPOD.IO/PRICING · COMMUNITY CLOUD · CAPTURED 16.10.2026



Green: RTX 4090, 24 GB — $0.34 per hour

ONE WORKING DAY OF A SUPPORT BOT

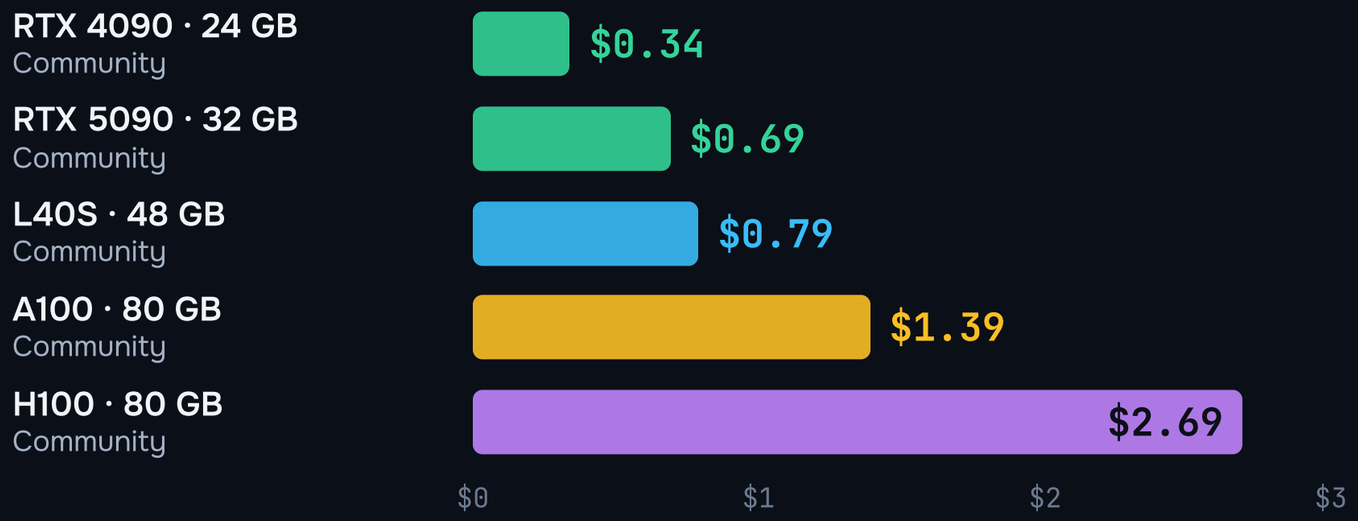


> **Trap:** a stopped Pod (Stop) still bills for its disk. When you are done, delete it (Terminate).

PRICE

Renting a GPU only pays off if it is busy all the time

RunPod Pods, price per hour, 16.10.2026



$0.73 → $7.30

per 1M tokens on an RTX 4090 (8B model, ≈130 tokens/s): busy **100%** of the time → busy **10%**

$0.40–4.50

per 1M output tokens for cheap cloud models (Gemini Flash-Lite, GPT mini)

> An idle GPU costs as much as a busy one. The reason for your own model is usually privacy and control, not savings.

HOW TO CHOOSE

Choose by two questions: can the data leave, and how much work is there

	Few requests	Many requests
Data may go to the model vendor	**Cloud API** ChatGPT, Gemini, Claude: best quality, pay per request	**Cloud API** the vendor’s batch processing is usually cheaper than renting
Data must stay with you	**Your laptop** LM Studio, Ollama	**RunPod** Pod for a steady flow; Serverless for bursts

> RunPod is also someone else’s computer, but the model and data are under your control, not the model vendor’s. Keep the code so the address changes with one line.

HOMEWORK

Homework: run a model yourself and compare it honestly with the cloud

1. Install LM Studio or Ollama; download a 3–9B model at Q4
2. Give it 3 work tasks, and the same tasks to ChatGPT or Gemini
3. Record the speed (tokens/s) and the memory the model uses
4. Score the answers 1–5 and draw a conclusion
5. Fill the card: API / laptop / Pod / Serverless and why

```
L10_homework/
├─ README.md          # choice and conclusion
├─ results.md         # 3 tasks: local vs cloud
├─ speed.png          # screenshot with tokens/s
└─ decision.md        # decision card
```

> No suitable computer? Take the smallest model (Granite 4.2 3B, 2.2 GB); it runs without a GPU.

Synthetic or your own data only. RunPod is optional and paid.

SUMMARY

Lesson summary

Open model = file + licence
find it on Hugging Face, read the card

Q4 + 25% must fit the GPU
otherwise it is slow; without a GPU take 3–4B

LM Studio or Ollama → localhost
one line changes in code; never expose the server

RunPod: idle time costs money
when done — Terminate, not Stop

> **Next, L11:** function calling and your first agent: the model not only answers but acts.